

Report Progetto: Web Parser ed Evaluator con LLM-as-a-Judge

Andrea Cerone
2126029

Alessandro Cordischi
2019924

Andrea Giordano
2148782

Sapienza Università di Roma
Laboratorio di Ingegneria Informatica

1 Obiettivo del Progetto

Il progetto consiste nello sviluppo di una complessa piattaforma software full-stack progettata per estrarre, pulire e valutare contenuti testuali provenienti da pagine web eterogenee. L'obiettivo primario è isolare l'informazione semantica rilevante (il contenuto principale), eliminando il rumore generato da boilerplate, menu di navigazione, script e banner pubblicitari. Il sistema non si limita tuttavia al solo web scraping, ma integra un rigoroso modulo di valutazione automatizzata che confronta il testo estratto con un Gold Standard di riferimento. Questa validazione incrociata avviene sfruttando un approccio ibrido: da un lato impiega metriche deterministiche a livello di token (Precision, Recall, F1-score) per l'allineamento sintattico, dall'altro utilizza un valutatore semantico basato su Intelligenza Artificiale (LLM-as-a-Judge) per comprendere le sfumature e la fedeltà contenutistica.

2 Organizzazione del Codice

L'architettura del sistema è costruita su rigidi principi di modularità e disaccoppiamento, ruotando attorno a un backend asincrono ad alte prestazioni sviluppato in FastAPI.

- **API e Validazione Dati (server.py):** Rappresenta il core asincrono dell'applicazione. Espone gli endpoint REST per l'interazione con il frontend e per la suite di grading. La robustezza degli input è garantita dai modelli Pydantic, che tipizzano rigidamente i payload in ingresso (es. `ParseRequest`, `JudgeRequest`), bloccando le richieste malformate prima dell'elaborazione.
- **Motore di Parsing (parsers/):** Segue un pattern architetturale Object-Oriented. Una classe astratta `BaseWebParser` definisce il contratto formale dei metodi, mentre le classi

figlie implementano logiche di estrazione specializzate. L'acquisizione delle pagine live è affidata al motore headless `Crawl4AI`, mentre la manipolazione dell'albero DOM è gestita tramite `BeautifulSoup`.

- **Modulo LLM Judge (judge.py):** Gestisce l'interazione con l'istanza locale del modello linguistico. Le chiamate HTTP verso il modello sono state implementate in modo strettamente asincrono tramite `httpx.AsyncClient` per evitare eccezioni di tipo `RuntimeError` e impedire che i lunghi tempi di inferenza blocchino l'Event Loop di FastAPI.
- **Frontend (templates/):** Sviluppato con Jinja2, HTML5 e CSS, offre una dashboard interattiva per lanciare parsing live o dal database. Consente la visualizzazione dinamica in affiancamento dell'HTML grezzo, dell'output in formato Markdown, del Gold Standard e delle metriche calcolate in tempo reale.

3 Valutazione Parser

L'estrazione del testo ha richiesto un intenso lavoro di data cleaning. La misurazione è stata condotta a livello di token: i testi vengono normalizzati tramite *lowercasing* ed estrazione puramente alfanumerica per calcolare le intersezioni insiemistiche. Complessivamente, il sistema ha raggiunto un F1-score medio aggregato eccellente (> 0.90) su tutto il dataset.

Mentre domini documentali come *Wikipedia* e *Scaruffi* hanno permesso l'adozione di logiche di estrazione dirette e lineari (raggiungendo facilmente F1-score prossimi a 0.99), le problematiche tecniche più complesse sono emerse nell'elaborazione del dominio *Spotify*, specialmente per i file HTML offline. Le pagine salvate nel database provenivano

spesso da Google Cache, che altera profondamente la struttura DOM rispetto alla pagina live. Mentre sul Live DOM è stato sufficiente intercettare selettori dinamici come `data-testid` (garantendo un altissimo F1), l'HTML in cache presentava un rumore massiccio derivante dalla ripetizione ciclica del nome dell'artista, da contatori numerici spuri e da timestamp delle durate non previsti dal Gold Standard. Il problema è stato risolto adottando un'**architettura ibrida a smistamento**: per le pagine in cache è stata implementata una vera e propria macchina a stati che, dopo aver rimosso il boilerplate tramite CSS, itera riga per riga per individuare i numeri di traccia validi, concatena i titoli ed esclude attivamente le metriche temporali e gli anni di pubblicazione.

4 Valutazione dei Judge

Per simulare il giudizio umano è stato selezionato il modello **llama3.2:3b** in esecuzione locale tramite Ollama. La scelta è ricaduta su questo LLM per via della sua velocità ottimale in ambienti CPU-bound, garantendo tempi di latenza accettabili, e per la sua spiccata affidabilità nel generare output strutturati. Per garantire una valutazione imparziale e deterministica, l'inferenza è stata vincolata con una temperature pari a 0.0 e un limite massimo di generazione (`num_predict`).

L'ingegnerizzazione del prompt ha previsto una netta divisione: il *System Prompt* assegna il ruolo di valutatore esperto, istruendo l'LLM a penalizzare l'inclusione di boilerplate e artefatti di navigazione; lo *User Prompt* riceve i testi, che vengono preventivamente sottoposti a una funzione di troncamento cautelativo (fissata a un limite di 4000 caratteri) per proteggere la *context window* del modello e prevenire allucinazioni semantiche. Sebbene l'API sia stata istruita a restituire un formato JSON, i modelli di piccola taglia tendono talvolta a includere delimitatori Markdown. Per blindare la pipeline, è stato inserito uno strato di validazione basato su espressioni regolari capace di "pulire" la stringa grezza ed estrarre il payload prima di invocare il deserializzatore JSON.

Al fine di rendere il sistema *fault-tolerant* e indipendente dall'ambiente di esecuzione, il modulo integra un algoritmo di **Multi-Host Probing**, il sistema tenta la connessione in cascata su molteplici endpoint. Qualora il container Ollama risultasse irraggiungibile o l'API andasse in timeout, il backend intercetta l'eccezione calcolando istantanea-

mente un **fallback euristico**: un punteggio simulato, e proiettato da 1 a 5, basato sulla copertura dei token tra testo estratto e Gold Standard.

5 Organizzazione del DB

Il layer di persistenza è stato implementato utilizzando MariaDB, una scelta strategica dettata dalla necessità di disporre di un database relazionale robusto e strutturato. Questa architettura client-server nativa si rivela ideale per gestire in modo efficiente ed efficientato pool di connessioni simultanee e transazioni complesse senza incorrere in colli di bottiglia prestazionali, specialmente durante le fasi di mass-evaluation parallela e intensiva. L'orchestrazione del server MariaDB avviene tramite un servizio containerizzato e isolato all'interno della rete di Docker Compose, garantendo persistenza sicura tramite volumi dedicati e totale indipendenza logica rispetto al codice del backend.

Il database è strutturato in modo rigorosamente normalizzato per garantire un perfetto isolamento tra i dati grezzi estratti in tempo reale e i dati di riferimento della verità di base. La tabella `web_resources` utilizza l'URL in formato stringa come chiave primaria e archivia il dominio di appartenenza, il titolo della pagina e l'intero dump dell'HTML grezzo acquisito in fase di crawling. Parallelamente, la tabella `gold_standard` utilizza a sua volta l'URL come chiave primaria, archiviando il testo di riferimento finale, validato manualmente, ripulito da artefatti e strutturato correttamente. L'architettura prevede inoltre operazioni CRUD complete ed atomiche per l'inserimento, l'aggiornamento e la rimozione sicura a cascata delle risorse.

Questa infrastruttura si rivela il vero motore logico che abilita la modalità operativa `local=True` esposta dall'endpoint `/parse`. In questa modalità, l'applicazione recupera il payload HTML archiviato in precedenza, bypassando completamente le latenze di rete e i vincoli del modulo di web scraping live. Ciò permette di rieseguire l'intera pipeline di estrazione testuale in frazioni di secondo, offrendo una funzionalità cruciale per le operazioni di valutazione aggregata e per il calcolo automatizzato dell'F1-score. Infine, l'uso di indici ottimizzati e chiavi primarie condivise facilita le operazioni relazionali e le JOIN necessarie per generare i report statistici complessi esposti dall'endpoint `/db_stats`.